

FLEETFLOW

SOFTWARE DESIGN DOCUMENT

Transport Operations Management System

Contents

- 1. Design Overview
- 2. Design Objectives and Scope
- 3. System Context and Architectural Design
- 4. Module Design
- 5. Database Design
- 6. API and Service Design
- 7. Business Rules and State Management
- 8. User Interface Design
- 9. Validation, Error Handling and Security Design
- 10. Non-Functional Design
- 11. Deployment and Configuration Design
- 12. Testing and Verification Strategy
- 13. Traceability and Design Consistency
- 14. Conclusion
- Appendix A. Key Data Dictionary
- Appendix B. Design Assumptions

1. Design Overview

FleetFlow is proposed as a web-based transport operations management system that centralizes vehicle, driver, trip, maintenance, fuel, expense and operational-monitoring activities. The system is intended to replace fragmented manual records with a single role-aware application in which operational information can be entered, validated, searched and viewed from a common interface.

The design adopts a layered client-server architecture. A React/Vite frontend will provide the user interface and navigation, an Express/Node.js backend will expose REST-style JSON services and enforce application rules, and PostgreSQL will provide persistent storage. The system will be organized into domain modules so that each major fleet activity can evolve independently while sharing authentication, authorization, validation and database infrastructure.

The design is derived from the FleetFlow Software Requirements Specification, the project structure supplied for this assignment, and the module boundaries already chosen for the project. The emphasis is therefore on proposed structure, responsibilities, interfaces, data models and quality attributes rather than on reporting completed implementation work.

1.1 Selected Technology Stack

Layer	Technology / Choice	Design Purpose
Frontend	React 18 + Vite	Component-based responsive user interface and fast development workflow
Routing	React Router	Protected navigation and role-aware page access
Client validation	Zod	Schema-based validation for authentication and domain forms
HTTP client	Axios	Centralized API communication and authentication headers
Charts / metrics	Recharts	Dashboard visualization of fleet and financial indicators
Backend	Node.js + Express 5	REST API, middleware pipeline and domain services
Authentication	JWT + bcrypt	Stateless access tokens with hashed passwords
Database	PostgreSQL	Relational persistence, constraints and transactions
Configuration	Environment variables	Separation of runtime configuration from source code

2. Design Objectives and Scope

2.1 Design Objectives

- Provide a single operational view of fleet assets, drivers and trips.
- Prevent invalid dispatches by checking vehicle capacity, resource availability and driver licence validity.
- Keep related operational records consistent when a business action changes more than one entity.
- Use role-based access control to expose only the functions relevant to each user class.
- Provide a simple REST API boundary so the user interface remains independent of database implementation details.
- Deliver clear validation, status indicators and error feedback to support safe day-to-day operations.

- Design the solution so that additional fleet functions can be added as modules without redesigning the complete application.

2.2 Functional Scope

Functional Area	Scope in the Design	Primary Roles
Authentication	Registration, login, current-user session and role selection	All
Vehicle Management	Create, view, filter, update and retire/delete fleet vehicles	Fleet Manager
Driver Management	Register drivers, maintain licence/safety information and identify eligible drivers	Fleet Manager, Safety Officer
Trip Management	Create draft trips, dispatch, complete and cancel trips	Fleet Manager, Driver
Maintenance	Open and close maintenance records and update vehicle maintenance state	Fleet Manager, Safety Officer
Fuel	Record fuel consumption and cost associated with vehicles/trips	Fleet Manager, Driver
Expenses	Record and review operating expenses and categories	Fleet Manager, Financial Analyst
Dashboard	KPIs, active trip monitoring, vehicle availability and profitability indicators	All authorized users

2.3 Out of Scope for the Design Stage

The present assignment does not require a production deployment, real-world telematics integration, payment processing, GPS hardware integration, mobile-native applications or external accounting-system integration. These may be considered future extensions after the core system has been developed and tested.

3. System Context and Architectural Design

3.1 Architectural Style

FleetFlow will use a three-layer logical architecture: presentation, application/business logic, and persistence. The presentation layer will contain React pages and reusable components. The application layer will contain Express routes, middleware, controllers, services and repositories grouped by domain. The persistence layer will contain the PostgreSQL database and transactional operations. This separation will allow the UI to change without directly exposing SQL details and will keep domain rules close to the business services that enforce them.

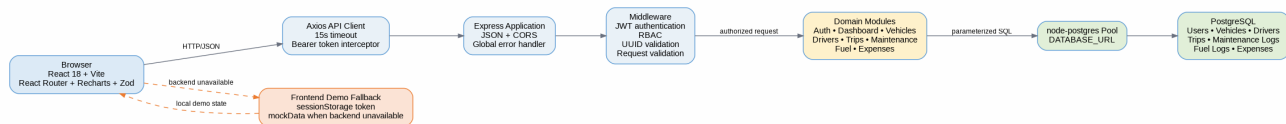


Figure 1. Proposed FleetFlow logical architecture and request flow

3.2 Frontend Architecture

The frontend will be structured around a shared application shell and page-oriented modules. App.jsx will compose routing and global providers; AuthContext will maintain authenticated-user state; AppRoutes will define page routes; ProtectedRoute and RoleRoute will guard access; DashboardLayout will provide shared navigation and top-bar elements. Domain API wrappers will isolate HTTP calls from page components. Reusable components such as DataTable, KpiCard, StatusBadge, PageHeader, ModuleToolbar and Modal will provide consistency across modules.

3.3 Backend Architecture

The backend will be organized into authentication, vehicle, driver, trip, maintenance, fuel, expense and dashboard modules. Each module will expose routes and controllers, while the modules that require richer business rules will use service and repository layers. Shared middleware will handle JWT authentication, role authorization and UUID parameter validation. This structure matches the intended package organization in the project and provides a clear separation of concerns for future implementation.

3.4 Request Flow

1. The user selects an action or opens a page in the browser.
2. A page-level API wrapper sends an HTTP request through the centralized Axios client.
3. The request includes the JWT bearer token when the operation requires authentication.
4. Express routes direct the request to the relevant controller after middleware checks.
5. The controller validates input and invokes the domain service/repository as appropriate.
6. The service performs business-rule checks and database operations, using a transaction for multi-record state changes.
7. The backend returns a JSON success or error response.
8. The frontend updates the relevant view and presents validation, loading or error feedback to the user.

4. Module Design

Module	Design Responsibility	Primary Interface
4.1 Authentication and Authorization	Manages user registration, login, authenticated-session lookup and role-based access. Passwords will be hashed with bcrypt and authentication responses will contain a signed JWT. The token will carry the minimum identity information required for authorization.	POST /api/auth/register; POST /api/auth/login; GET /api/auth/me
4.2 Vehicle Management	Maintains fleet asset information such as registration number, vehicle name, model, type, region, capacity, odometer, acquisition cost and status. Vehicle status will control whether the vehicle can be dispatched, placed in maintenance or retired.	GET/POST /api/vehicles; GET/PUT/DELETE /api/vehicles/:id
4.3 Driver Management	Maintains driver licence, category, expiry date, phone, safety score and availability. Driver eligibility for dispatch will depend on status and licence validity.	GET /api/drivers; GET /api/drivers/available; POST /api/drivers
4.4 Trip Management	Represents the main operational workflow from trip creation to dispatch, completion or cancellation. Trip creation will capture vehicle, driver, route, cargo, distance and revenue; dispatch and completion will update related resource state.	GET/POST /api/trips; POST /api/trips/:id/dispatch; POST /api/trips/:id/complete; POST /api/trips/:id/cancel
4.5 Maintenance Management	Records maintenance work, its type, description, cost, start date and completion status. Opening maintenance will move the related vehicle into an unavailable state; closure will return an eligible vehicle to availability.	GET/POST /api/maintenance; PUT /api/maintenance/:id/close
4.6 Fuel Management	Records fuel quantity and cost against a vehicle and optionally a trip. Each fuel entry will have a corresponding expense representation so operational cost reporting can remain consistent.	GET/POST /api/fuel
4.7 Expense Management	Stores manual and system-generated operational expenses with category, amount, description, date and optional vehicle/trip association. Categories will include Tolls, Parking, Maintenance, Fuel, Insurance and Other.	GET/POST /api/expenses
4.8 Dashboard and Analytics	Aggregates operational and financial indicators such as active trips, available vehicles, vehicles in shop, completed-trip revenue, expenses, net profit and per-vehicle ROI.	GET /api/dashboard

- Deleting a vehicle that participates in an active trip or active maintenance record shall be prohibited.

5.3 Indexing Strategy

Indexes will be created on registration_no, email, license_no, status fields, vehicle_id, driver_id, trip_id and date fields frequently used by filters, joins or dashboard aggregates. The precise index set may be tuned after profiling realistic workloads.

6. API and Service Design

6.1 API Conventions

The API will use HTTP/HTTPS with JSON request and response bodies. Protected endpoints will use the Authorization header with the Bearer scheme. Responses will follow a consistent structure using a success indicator and either returned data or a structured error object. Validation failures will use HTTP 400, authentication failures HTTP 401, authorization failures HTTP 403, missing resources HTTP 404 and unexpected server failures HTTP 500.

6.2 Endpoint Catalogue

Module	Method	Endpoint	Purpose	Access
Auth	POST	/api/auth/register	Create account	Public
Auth	POST	/api/auth/login	Authenticate user	Public
Auth	GET	/api/auth/me	Get current user	Authenticated
Dashboard	GET	/api/dashboard	Get dashboard KPIs	Authenticated
Vehicles	GET	/api/vehicles	List/filter vehicles	Authenticated
Vehicles	POST	/api/vehicles	Create vehicle	Fleet Manager
Vehicles	GET	/api/vehicles/:id	Get one vehicle	Authenticated
Vehicles	PUT	/api/vehicles/:id	Update vehicle	Fleet Manager
Vehicles	DELETE	/api/vehicles/:id	Delete/retire vehicle	Fleet Manager
Drivers	GET	/api/drivers	List drivers	Authenticated
Drivers	GET	/api/drivers/available	List eligible drivers	Authenticated
Drivers	POST	/api/drivers	Create driver	Fleet Manager / Safety Officer
Trips	GET	/api/trips	List trips	Authenticated
Trips	POST	/api/trips	Create trip	Fleet Manager / Driver
Trips	POST	/api/trips/:id/dispatch	Dispatch trip	Fleet Manager / Driver
Trips	POST	/api/trips/:id/complete	Complete trip	Fleet Manager / Driver
Trips	POST	/api/trips/:id/cancel	Cancel trip	Fleet Manager / Driver
Maintenance	GET	/api/maintenance	List maintenance logs	Authenticated
Maintenance	POST	/api/maintenance	Open maintenance	Fleet Manager / Safety Officer
Maintenance	PUT	/api/maintenance/:id/close	Close maintenance	Fleet Manager / Safety Officer
Fuel	GET	/api/fuel	List fuel logs	Authenticated
Fuel	POST	/api/fuel	Create fuel log	Fleet Manager / Driver
Expenses	GET	/api/expenses	List expenses	Fleet Manager / Financial Analyst
Expenses	POST	/api/expenses	Create expense	Fleet Manager / Financial Analyst

6.3 Transactional Service Operations

Workflow	Design Transaction Sequence
Dispatch trip	Begin transaction → lock trip → verify Draft → lock vehicle and driver → verify availability and driver eligibility → update trip to Dispatched → set vehicle/driver to On Trip → commit
Complete trip	Begin transaction → lock trip → verify Dispatched → record actual distance/end odometer/completion time → set vehicle to Available and update odometer → set driver to Available → commit
Cancel trip	Begin transaction → lock trip → if necessary restore vehicle and driver availability → mark trip Cancelled → commit
Open maintenance	Begin transaction → lock vehicle → verify vehicle is not On Trip → create maintenance log → set vehicle In Shop → create Maintenance expense → commit
Fuel entry	Begin transaction → create fuel log → create Fuel expense → commit

7. Business Rules and State Management

7.1 Vehicle Rules

- A newly created vehicle will have status Available.
- A vehicle on a trip shall not be assigned to another trip.
- A vehicle in maintenance shall not be dispatched.
- A vehicle may be retired only when it is not participating in an active trip or maintenance operation.

7.2 Driver Rules

- A new driver will have status Available.
- A driver with an expired licence shall not be selected for dispatch.
- A driver already On Trip shall not receive another active trip.
- A Suspended driver shall not be eligible for dispatch.

7.3 Trip Rules

- New trips start in Draft state.
- Dispatch changes a valid Draft trip to Dispatched and reserves the selected vehicle and driver.
- Completion is permitted only for a Dispatched trip and requires actual distance and ending odometer.
- Cancellation may occur from Draft or Dispatched; if the trip was active, reserved resources must be released.
- Cargo weight must not exceed vehicle maximum load capacity.

7.4 State Transition Design

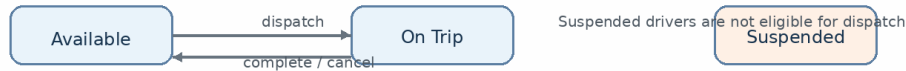
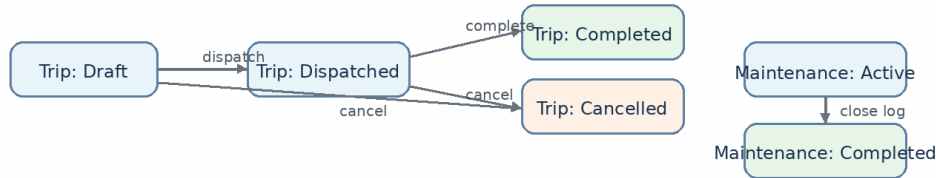
Vehicle State**Driver State****Trip and Maintenance States**

Figure 3. Proposed vehicle, driver and trip state transitions

Entity	States	Important Transitions
Vehicle	Available, On Trip, In Shop, Retired	Available → On Trip → Available; Available → In Shop → Available; Available → Retired
Driver	Available, On Trip, Suspended	Available → On Trip → Available; Suspended blocks dispatch
Trip	Draft, Dispatched, Completed, Cancelled	Draft → Dispatched → Completed; Draft/Dispatched → Cancelled
Maintenance Log	Active, Completed	Active → Completed

8. User Interface Design

8.1 UI Design Principles

- Use a consistent dashboard shell with sidebar navigation, top bar and page-level headers.
- Use tables for operational records and modal forms for focused create/update actions.
- Use status badges for state visibility and consistent terminology.
- Provide inline validation and clear empty/loading/error states.
- Use responsive grids so the main screens remain usable on laptop and smaller screens.
- Keep role-specific navigation simple so users see only the functions relevant to their responsibilities.

8.2 Screen Specifications

Screen	Main Elements	Primary Interactions
Login / Sign Up	Branding, sign-in/sign-up tabs, email, password, role selection	Authenticate or create account
Dashboard	KPI cards, active trip watchlist, vehicle availability, revenue/expense indicators	View operational status and financial summary
Vehicles	Toolbar, search/filter controls, vehicle data table, add/edit modal	Create, inspect, update and retire/delete vehicles

Drivers	Driver table, licence/safety fields, add driver modal	Register drivers and review eligibility
Trips	Trip table, route/cargo details, create trip form, status actions	Create, dispatch, complete or cancel trips
Maintenance	Maintenance table, add-maintenance modal, close action	Open and close maintenance records
Fuel	Fuel-log table and entry form	Record and review fuel transactions
Expenses	Expense table and entry form	Record and review costs
Forbidden / Not Found	Clear explanatory message and navigation action	Recover from invalid route or unauthorized access

8.3 Role-Oriented Navigation

Role	Primary Pages	Typical Responsibilities
Fleet Manager	Dashboard, Vehicles, Drivers, Trips, Maintenance, Fuel, Expenses	Fleet-wide planning, resource allocation, operational control and cost review
Driver	Dashboard, Trips, Fuel	Execute assigned trips and record trip-related fuel information
Safety Officer	Dashboard, Drivers, Maintenance	Driver compliance and maintenance/safety oversight
Financial Analyst	Dashboard, Expenses	Expense and profitability analysis

8.4 Reusable Components

Component	Responsibility
DataTable	Standardize tabular records, headers, loading and empty states
KpiCard	Display a label/value KPI pair
StatusBadge	Render operational state consistently
PageHeader	Provide title and page action area
ModuleToolbar	Search, filter and page-level controls
Modal	Reusable create/edit interaction container
InlineError	Display field or action errors close to their source
PageLoader	Standard loading feedback
AppErrorBoundary	Prevent one rendering error from breaking the whole interface

9. Validation, Error Handling and Security Design

9.1 Input Validation

Domain	Key Validation Rules
Authentication	Valid email and password; registration name/email/password/role required; password minimum 8 characters
Vehicle	Registration, name, type and region required; capacity > 0; odometer and acquisition cost >= 0
Driver	Licence number/category required; valid expiry date; phone length validated when provided; safety score 0–100
Trip	Vehicle/driver identifiers must be valid UUIDs; source/destination required; cargo and planned distance > 0; revenue >= 0
Maintenance	Vehicle identifier and maintenance type required; cost >= 0
Fuel / Expense	Vehicle and numeric values validated; amounts must be non-negative; category constrained to supported values

9.2 Error Handling

The backend will return structured error responses so the frontend can display meaningful feedback. Domain errors should be expressed with stable codes where useful, such as `VEHICLE_UNAVAILABLE`, `DRIVER_UNAVAILABLE`, `LICENSE_EXPIRED`, `CARGO_EXCEEDS_CAPACITY`, `INVALID_STATUS` and `NOT_FOUND`. The frontend will normalize API errors and provide inline or page-level feedback rather than silently failing.

9.3 Security Design

- Passwords will be hashed with bcrypt and never stored in plaintext.
- JWT tokens will be signed with a secret stored outside source code in an environment variable.
- Protected APIs will verify authentication before role checks or domain operations.
- Role-based authorization will be enforced on the backend; client-side route restrictions are only a usability layer.
- Database queries will use parameterized placeholders to reduce SQL-injection risk.
- Production deployment will use HTTPS, protected secrets, secure token handling, rate limiting and appropriate security headers.

10. Non-Functional Design

Quality Attribute	Design Target / Response
Performance	Use pooled PostgreSQL connections, indexed lookups, efficient SQL aggregates and avoid unnecessary API calls.
Reliability	Use transactions for multi-record state changes and rollback on failure.
Security	JWT authentication, RBAC, bcrypt, parameterized SQL, HTTPS in production and secure configuration.
Usability	Consistent layouts, status badges, clear validation messages, responsive design and role-focused navigation.
Maintainability	Domain-oriented backend modules, reusable frontend components, central API utilities and validation schemas.
Scalability	Keep frontend stateless and backend horizontally extensible; PostgreSQL remains the shared source of truth.
Testability	Separate validation, service and repository concerns so domain rules can be tested independently.
Availability	Use health checks, connection error handling and graceful application-level failure messages in deployed environments.

11. Deployment and Configuration Design

11.1 Logical Deployment

The development environment will contain three logical processes: a Vite development server for the client, an Express server for the API and a PostgreSQL database instance. In a production environment the React application can be built into static assets and served through a web server or CDN, while the Express API runs as an application service connected to PostgreSQL.

11.2 Configuration Variables

Variable	Layer	Purpose
<code>VITE_API_URL</code>	Frontend	Optional API base URL; defaults to <code>/api</code> during development
<code>DATABASE_URL</code>	Backend	PostgreSQL connection string

JWT_SECRET	Backend	Secret used to sign and verify JWTs
------------	---------	-------------------------------------

11.3 Deployment Flow

1. Configure environment variables and database connection.
2. Provision PostgreSQL tables, constraints and indexes from the approved database design.
3. Start the backend only after database connectivity has been verified.
4. Build/start the frontend with the API base URL configured.
5. Use HTTPS and protected secret storage in production.

12. Testing and Verification Strategy

Testing will be planned alongside development so that every major business rule has a corresponding verification case. The test strategy will cover unit-level validation, service-level business rules, API integration and end-to-end UI workflows.

Test Level	Representative Tests	Expected Outcome
Unit	Schema validation, formatting, status helpers, utility functions	Correct inputs accepted and invalid inputs rejected
Service	Dispatch eligibility, capacity check, licence expiry, state transitions	Business rules enforced independent of UI
API Integration	Authentication, CRUD endpoints, authorization, transaction workflows	Correct HTTP status and response structure
Database	Foreign keys, uniqueness, transaction rollback, concurrent dispatch	Data integrity maintained
UI / System	Login, role navigation, create vehicle/driver/trip, maintenance, fuel and expense workflows	User can complete intended workflow with clear feedback
Non-functional	Response time sampling, security review, responsive layout checks	System satisfies agreed quality targets

12.1 Critical Acceptance Scenarios

- Two simultaneous attempts to dispatch the same vehicle/driver must not both succeed.
- A driver with an expired licence must not be dispatched.
- A trip carrying more than vehicle capacity must be rejected.
- Opening maintenance for an On Trip vehicle must be rejected.
- Completing a trip must release the driver and vehicle and update the vehicle odometer.
- Cancelling an active trip must release reserved resources.
- Fuel and maintenance entries must be reflected in expense-based cost calculations.

13. Traceability and Design Consistency

The following matrix provides a direct consistency check between the Lab 3 design and the project structure. The wording is intentionally design-oriented: file and module names identify where each planned responsibility belongs, without claiming that the final functional implementation already exists.

Design Element	Project Structure / Naming Basis	Consistency
React/Vite frontend	client/, client/src/App.jsx, client/src/pages, client/src/components	Aligned
Express/Node backend	server/, server/src/app.js, server/src/server.js	Aligned

Domain modules	server/src/modules/auth, vehicle, driver, trip, maintenance, fuel, expense, dashboard	Aligned
Frontend API layer	client/src/api/* and client/src/services/api.js	Aligned
Authentication context	client/src/context/AuthContext.jsx	Aligned
Protected routing	client/src/routes/ProtectedRoute.jsx and RoleRoute.jsx	Aligned
Validation schemas	client/src/schemas/* and backend module validation files	Aligned
Reusable UI	client/src/components/*	Aligned
PostgreSQL persistence	database/schema.sql and server/src/config/db.js	Aligned as the planned persistence layer
Project naming	FleetFlow is the assignment/project name; legacy TransitOps identifiers remain only as historical scaffold names	Design uses FleetFlow consistently

13.1 Design Decisions Used to Avoid Ambiguity

- The product name in this document is always FleetFlow.
- The four intended user classes are Fleet Manager, Driver, Safety Officer and Financial Analyst.
- The operational states are written exactly as: Available, On Trip, In Shop, Retired, Draft, Dispatched, Completed and Cancelled.
- Fuel is treated as an operational record that also contributes to the Expense ledger for cost reporting.
- The database model is presented as a logical design to be implemented during the development stage; this document does not claim that the final database has already been provisioned.

14. Conclusion

The FleetFlow Software Design Document defines a coherent technical blueprint for the next SDLC stage of the project. It specifies the proposed architecture, domain modules, relational data model, API boundary, state transitions, UI structure, validation rules, security controls, non-functional goals, deployment approach and verification strategy.

The design is deliberately positioned as a pre-implementation artifact. It provides enough technical detail for development to proceed systematically while preserving a clear separation between requirements/design and the later implementation and testing stages. The module names, workflows, user roles, state terminology and technology choices remain consistent with the FleetFlow project materials supplied for the assignment.

Appendix A. Key Data Dictionary

Field	Type (Logical)	Description
users.id	UUID	Unique user identifier
users.email	VARCHAR	Unique login email
users.role	ENUM/VARCHAR	Fleet Manager / Driver / Safety Officer / Financial Analyst
vehicles.registration_no	VARCHAR	Unique vehicle registration identifier
vehicles.vehicle_type	VARCHAR	Vehicle category/type
vehicles.max_load_capacity	NUMERIC	Maximum supported cargo weight
vehicles.odometer	NUMERIC	Current odometer reading
vehicles.status	VARCHAR	Available / On Trip / In Shop / Retired
drivers.license_no	VARCHAR	Driver licence number
drivers.license_category	VARCHAR	Licence class/category
drivers.license_expiry	DATE	Licence expiry date
drivers.safety_score	NUMERIC	Safety rating from 0 to 100
drivers.status	VARCHAR	Available / On Trip / Suspended
trips.cargo_weight	NUMERIC	Cargo assigned to trip
trips.planned_distance	NUMERIC	Estimated trip distance
trips.actual_distance	NUMERIC	Actual distance after completion
trips.revenue	NUMERIC	Revenue attributed to trip
trips.status	VARCHAR	Draft / Dispatched / Completed / Cancelled
maintenance_logs.cost	NUMERIC	Maintenance cost
fuel_logs.liters	NUMERIC	Fuel quantity
fuel_logs.cost	NUMERIC	Fuel cost
expenses.amount	NUMERIC	Expense amount

Appendix B. Design Assumptions

- PostgreSQL is the selected persistent database for the project.
- UUIDs are used for entity identifiers.
- Role names and workflow states listed in this document are controlled vocabulary and should not be changed casually during implementation.
- The backend is the authoritative enforcement point for authorization and business rules; the frontend mirrors those restrictions for usability.
- The database schema will be created from the logical model before integrated testing begins.
- Production hardening items such as HTTPS, secure secret management, rate limiting, logging and security headers are deployment concerns beyond the initial academic prototype.
- External telematics, GPS and payment integrations are future extensions and are not required by the present design.